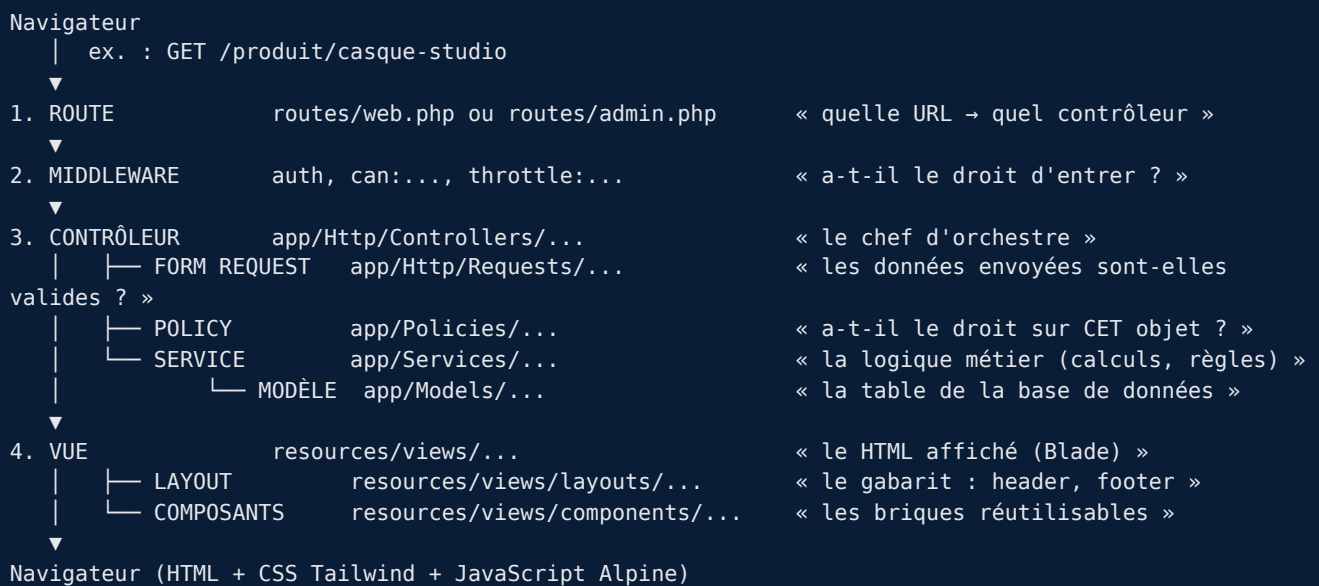


Guide du développeur — comprendre et modifier le projet

Ce guide explique **comment le code est organisé** et **comment modifier, ajouter ou supprimer une fonctionnalité**, avec des exemples pas à pas. Il complète [ARCHITECTURE.md](#), plus technique.

1. Le principe de base : le trajet d'une requête

Quand quelqu'un ouvre une page ou clique sur un bouton, la demande suit toujours le même chemin. Si vous retenez ce schéma, vous saurez toujours où chercher :



Méthode pour trouver n'importe quel code : partez de l'URL. Cherchez-la dans `routes/`, cela vous donne le contrôleur ; le contrôleur vous donne la vue et le service.

Astuce : `php artisan route:list` affiche toutes les URL avec leur contrôleur. Exemple : `php artisan route:list --path=panier`

2. Carte des dossiers

digit/	
├─ app/	← TOUT le code PHP de l'application
│ ├─ Assistant/	Chatbot optionnel (contrat + réponses automatiques)
│ ├─ Console/Commands/	Commandes artisan maison (shop:create-admin)
│ ├─ Delivery/	Livreurs (bouton « Contacter un livreur »)
│ ├─ Enums/	Listes de valeurs fixes : statuts de commande, rôles...
│ ├─ Events/ + Listeners/	« Quand X arrive, faire Y » (ex. commande créée → e-mail)
│ ├─ Exceptions/	BusinessException = erreur affichée proprement au client
│ ├─ Http/	
│ │ └─ Controllers/	
│ │ ├─ Shop/	Pages publiques : accueil, catalogue, produit, panier, commande
│ │ │ └─ Account/	Espace client : mes commandes, adresses, favoris...
│ │ │ └─ Admin/	Back-office
│ │ │ └─ Auth/	Connexion, inscription, mot de passe oublié
│ │ └─ Middleware/	Filtres appliqués à chaque requête (compte actif, sécurité)
│ │ └─ Requests/	Règles de validation des formulaires
│ └─ Models/	Une classe = une table (Product, Order, User...)
│ └─ Notifications/	E-mails envoyés aux clients
│ └─ Payments/	Moyens de paiement (un fichier par moyen)
│ └─ Policies/	Qui a le droit de voir/modifier quel objet
│ └─ Providers/	Démarrage de l'appli (droits, limites de requêtes...)
│ └─ Services/	★ LA LOGIQUE MÉTIER (panier, commande, paiement, stock)
│ └─ Support/	Petits outils (Money = formatage des prix)
│ └─ helpers.php	Fonctions globales : money(), setting()
├─ config/	Réglages lus depuis .env
│ └─ shop.php	Nom, devise, contact, seuils...
│ └─ payments.php	Liste des moyens de paiement
│ └─ delivery.php	Livreur
│ └─ permissions.php	★ Ce que chaque rôle a le droit de faire
│ └─ assistant.php	Chatbot
├─ database/	
│ └─ migrations/	★ Structure des tables (création/modification)
│ └─ factories/	Fabriques de fausses données (pour les tests)
│ └─ seeders/	Remplissage de la base (rôles, données de démo)
├─ resources/	
│ └─ css/	
│ │ └─ theme.css	★ Couleurs et polices du site (UN seul endroit)
│ │ └─ components.css	Styles des boutons, champs, cartes, badges
│ │ └─ app.css	Feuille de la boutique
│ │ └─ admin.css	Feuille de l'administration
│ └─ js/	JavaScript (Alpine.js) : panier AJAX, recherche, toasts...
│ └─ views/	★ Les pages HTML (Blade)
│ │ └─ layouts/	Gabarits : shop (boutique), account, admin, auth
│ │ └─ partials/	Morceaux de layout : header, footer, recherche
│ │ └─ components/	Briques : <x-product-card>, <x-icon>, <x-form.input>...
│ │ └─ shop/	Pages publiques
│ │ └─ account/	Espace client
│ │ └─ admin/	Back-office
│ │ └─ auth/	Connexion / inscription
│ │ └─ errors/	Pages 404, 500...
├─ routes/	
│ └─ web.php	★ URL publiques + espace client
│ └─ admin.php	★ URL de l'administration
│ └─ console.php	Tâches planifiées (expiration des paiements)
└─ lang/fr/	Messages d'erreur de validation en français

tests/	Tests automatiques (Feature = scénarios, Unit = calculs)
public/	Seul dossier visible sur internet (images, CSS compilé)
storage/	Fichiers générés : images uploadées, logs, cache
.env	★ Configuration de VOTRE machine (jamais sur GitHub)
.env.example	Modèle du .env

Les ★ marquent les fichiers que vous toucherez le plus souvent.

3. Qui fait quoi : les règles du projet

Couche	Rôle	À ne PAS faire
Route	Associe une URL à une méthode de contrôleur, applique les protections	Pas de logique dedans
Contrôleur	Reçoit la requête, appelle validation, autorisation et service, renvoie une vue	Pas de calculs métier (prix, stock)
Form Request	Valide les champs du formulaire	—
Policy	Répond à « cet utilisateur peut-il agir sur cet objet ? »	—
Service	Les règles métier : calcul du total, contrôle du stock, changement de statut	—
Modèle	Représente une table : relations, conversions de types	Pas de HTML
Vue Blade	Affiche les données	Pas de requêtes SQL ni de calculs métier

Trois règles spécifiques au projet :

1. **Les prix sont des entiers** (12 500 FCFA = 12500). Pour afficher : `{{ money($prix) }}`.
2. **Un statut de commande ne se change JAMAIS à la main** (`$order->status = ...`). On passe par `OrderStatusService::transition()`, qui vérifie que le changement est autorisé, remet le stock si besoin, écrit l'historique et prévient le client.
3. **Les couleurs ne s'écrivent jamais en dur** (`#ff6b35`) : on utilise les classes `bg-brand-900`, `text-accent-700` ... définies dans `theme.css`.

4. Exemple guidé : suivre une fonctionnalité existante

« Ajouter au panier », du clic jusqu'à la base :

1. **La vue** `resources/views/shop/product.blade.php` contient le bouton `@click="add()"`.
2. **Le JavaScript** `resources/js/components.js` (bloc `productPurchase`) envoie `POST /panier/articles` et `resources/js/stores.js` (`Alpine.store('cart')`) met à jour le compteur du panier.
3. **La route**, dans `routes/web.php` : `Route::post('/articles', [Shop\CartController::class, 'store'])`.
4. **La validation** `app/Http/Requests/Shop/AddToCartRequest.php` : produit existant, quantité entre 1 et 20.
5. **Le contrôleur** `app/Http/Controllers/Shop/CartController.php` → méthode `store()`, qui appelle le service.
6. **Le service** `app/Services/Cart/CartService.php` → méthode `add()` : produit publié ? variante valide ? stock suffisant ?
7. **Les modèles** `Cart` et `CartItem` (`app/Models/`) enregistrent la ligne en base.
8. La réponse JSON revient et le mini-panier s'ouvre.

Faites le même exercice avec « Passer commande » : `CheckoutController@store` → `OrderService::placeOrder()` → `PaymentService::start()`.

5. Recettes : MODIFIER, AJOUTER, SUPPRIMER

Méthode générale (à appliquer à chaque demande)

1. **Comprendre** la demande et trouver le code concerné (méthode de la section 1, ou une recherche dans tout le projet : `Ctrl+Maj+F` dans VS Code).
 2. **Lister les couches touchées** : base de données ? validation ? logique ? affichage ? droits ?
 3. **Modifier dans l'ordre** : migration → modèle → validation → service → contrôleur → vue.
 4. **Tester** à la main dans le navigateur, puis `php artisan test`.
 5. **Enregistrer** avec Git (section 7).
-

Recette A — MODIFIER : ajouter un champ « Marque » aux produits

Demande du client : « Je veux indiquer la marque de chaque produit et pouvoir la rechercher. »

Étape 1 — Base de données. On ne modifie jamais une ancienne migration déjà exécutée : on en crée une nouvelle.

```
php artisan make:migration add_brand_to_products_table
```

Ouvrez le fichier créé dans `database/migrations/` :

```
public function up(): void
{
    Schema::table('products', function (Blueprint $table) {
        $table->string('brand', 120)->nullable()->after('name');
    });
}

public function down(): void
{
    Schema::table('products', function (Blueprint $table) {
        $table->dropColumn('brand');
    });
}
```

Puis : `php artisan migrate`

Étape 2 — Modèle. Dans `app/Models/Product.php`, ajoutez `'brand'` à la liste `#[Fillable(...)]`. Sans cela, Laravel ignore le champ, par sécurité.

Étape 3 — Validation. Dans `app/Http/Requests/Admin/ProductRequest.php`, méthode `rules()` :

```
'brand' => ['nullable', 'string', 'max:120'],
```

Étape 4 — Formulaire admin. Dans `resources/views/admin/products/form.blade.php`, sous le champ « Nom » :

```
<x-form.input name="brand" label="Marque" :value="$product->brand" maxlength="120" />
```

Le service `ProductManager` enregistre automatiquement tous les champs validés : rien à changer.

Étape 5 — Affichage. Dans `resources/views/shop/product.blade.php`, sous le titre :

```
@if ($product->brand)
    <p class="mt-1 text-sm text-zinc-500">Marque : {{ $product->brand }}</p>
@endif
```

Étape 6 — Recherche (optionnel). Dans `app/Services/Catalog/ProductSearch.php`, méthode `applySearch()`, ajoutez une ligne à côté des autres :

```
$this->whereContains($q, 'brand', $word, 'or');
```

Étape 7 — Test. Dans `tests/Feature/Admin/ProductManagementTest.php`, ajoutez `'brand' => 'Sony'` au formulaire du premier test et vérifiez `$this->assertSame('Sony', $product->brand);`. Puis `php artisan test`.

Recette B — AJOUTER : une page « FAQ »

Le projet a déjà un système de pages d'information : il suffit de le suivre.

1. Dans `app/Http/Controllers/Shop/PageController.php`, ajoutez une entrée au tableau `PAGES` :

```
'faq' => ['title' => 'Questions fréquentes', 'description' => 'Les réponses à vos questions.'],
```

2. Créez `resources/views/shop/pages/faq.blade.php` (copiez `livraison.blade.php` comme modèle) :

```
@extends('shop.pages.layout')

@section('page')
    <h2>Quels sont les délais de livraison ?</h2>
    <p>24 à 72 h à Abidjan.</p>
@endsection
```

3. C'est tout. La route (`routes/web.php` accepte automatiquement les clés de `PAGES`), le lien dans le footer et le sitemap se mettent à jour seuls.

Pour une fonctionnalité complète et nouvelle (par exemple « Liste de cadeaux »), le schéma est toujours le même :

Étape	Commande / fichier
Table	<code>php artisan make:migration create_gift_lists_table</code>
Modèle	<code>php artisan make:model GiftList</code> → <code>app/Models/GiftList.php</code>
Validation	<code>php artisan make:request StoreGiftListRequest</code>
Logique	un fichier dans <code>app/Services/</code> si des règles métier existent
Contrôleur	<code>php artisan make:controller Account/GiftListController</code>
Droits	<code>php artisan make:policy GiftListPolicy --model=GiftList</code>
URL	une ligne dans <code>routes/web.php</code> (ou <code>routes/admin.php</code>)
Vue	un fichier dans <code>resources/views/account/</code> qui commence par <code>@extends('layouts.account')</code>
Menu	ajouter le lien dans <code>layouts/account.blade.php</code> (tableau <code>\$links</code>) ou <code>layouts/admin.blade.php</code> (tableau <code>\$nav</code>)
Test	<code>php artisan make:test GiftListTest</code>

Recette C — SUPPRIMER : retirer la newsletter

Règle d'or : avant de supprimer, **trouvez toutes les utilisations**. Dans VS Code, `Ctrl+Maj+F` et cherchez `newsletter` (ou dans le terminal : `git grep -n -i newsletter`).

Vous trouverez :

Où	Quoi faire
<code>resources/views/shop/home.blade.php</code>	Supprimer le bloc <code>{{-- Newsletter --}}</code>
<code>routes/web.php</code>	Supprimer les 2 routes <code>newsletter</code>
<code>routes/admin.php</code>	Supprimer les 2 routes <code>newsletter</code>
<code>resources/views/layouts/admin.blade.php</code>	Supprimer la ligne <code>Newsletter</code> du menu <code>\$nav</code>
<code>app/Http/Controllers/Shop/NewsletterController.php</code>	Supprimer le fichier
<code>app/Http/Controllers/Admin/NewsletterController.php</code>	Supprimer le fichier
<code>resources/views/admin/newsletter.blade.php</code>	Supprimer le fichier
<code>tests/Feature/Shop/StorefrontTest.php</code>	Supprimer le test <code>newsletter</code>
<code>app/Models/NewsletterSubscriber.php</code> + la table	Optionnel : la table peut rester ; pour la supprimer, créer une migration avec <code>Schema::dropIfExists('newsletter_subscribers')</code>

Puis vérifiez qu'il ne reste rien : refaites la recherche, puis `php artisan route:list` et `php artisan test`.

Plus prudent : **désactiver plutôt que supprimer**. Entourez le bloc de la page d'accueil de `@if (false) ... @endif`, ou mieux, créez un réglage dans `config/shop.php` (`'newsletter' => env('SHOP_NEWSLETTER', true)`) et testez `@if (config('shop.newsletter'))`. Le client peut alors changer d'avis sans redéveloppement.

Recette D — Modifications courantes et rapides

Demande	Où
Changer les couleurs	<code>resources/css/theme.css</code> , puis <code>npm run build</code>
Changer un texte de la page d'accueil	Admin > Paramètres > Contenus (sinon <code>resources/views/shop/home.blade.php</code>)
Changer un texte du header / footer	<code>resources/views/partials/header.blade.php</code> / <code>footer.blade.php</code>
Changer la carte produit (partout sur le site)	<code>resources/views/components/product-card.blade.php</code>
Ajouter une icône	<code>resources/views/components/icon.blade.php</code> (tableau <code>\$paths</code>), puis <code><x-icon name="..." /></code>
Donner un droit à un rôle	<code>config/permissions.php</code>
Changer un message d'erreur de validation	<code>lang/fr/validation.php</code> ou la méthode <code>messages()</code> du Form Request
Changer le contenu d'un e-mail	<code>app/Notifications/</code>
Changer les étapes du statut de commande	<code>app/Enums/OrderStatus.php</code> (méthode <code>allowedTransitions()</code>)
Changer le nombre de produits par page	<code>config/shop.php</code> → <code>per_page</code>
Ajouter un moyen de paiement	voir ARCHITECTURE.md , section Paiements

6. Les commandes à connaître

Commande	Utilité
<code>php artisan serve</code>	Lancer le site en local
<code>npm run dev</code>	Recompiler CSS/JS automatiquement pendant que vous codez (dans une 2 ^e fenêtre)
<code>npm run build</code>	Compiler CSS/JS pour la production
<code>php artisan migrate</code>	Appliquer les nouvelles migrations
<code>php artisan migrate:fresh --seed</code>	Tout effacer et recharger la démo (jamais en production)
<code>php artisan route:list</code>	Voir toutes les URL
<code>php artisan tinker</code>	Console pour tester du PHP sur la vraie base (<code>App\Models\Product::count()</code>)
<code>php artisan test</code>	Lancer les tests
<code>vendor/bin/pint</code>	Remettre le code au bon format
<code>php artisan optimize:clear</code>	Vider tous les caches (si une modification ne s'affiche pas)
<code>php artisan make:model / make:controller / make:migration / make:request / make:policy / make:test</code>	Générer des fichiers vides au bon endroit

7. Déboguer quand ça ne marche pas

1. **Lire l'erreur.** En local (`APP_DEBUG=true` dans `.env`), Laravel affiche une page avec le fichier et la ligne en cause.
 2. **Lire le journal :** `storage/logs/laravel.log` (le plus récent est en bas).
 3. **Une modification n'apparaît pas ?** CSS/JS : relancez `npm run build` ou laissez tourner `npm run dev`. Config ou routes : `php artisan optimize:clear`.
 4. **Inspecter une valeur :** écrivez `dd($variable);` dans le contrôleur. La page s'arrête et l'affiche. **Pensez à l'enlever ensuite.**
 5. **Erreur 403** = la Policy ou `config/permissions.php` refuse. **Erreur 419** = jeton CSRF : il manque `@csrf` dans le formulaire.
 6. **Erreur « Attempted to lazy load »** : une relation est chargée dans une boucle (problème de performance N+1). Ajoutez `->with('relation')` à la requête du contrôleur.
-

8. Travailler proprement avec Git

```
git checkout -b ajout-champ-marque      # une branche par demande
# ... modifications ...
php artisan test                        # tout doit être vert
vendor/bin/pint                        # mise en forme
git add -A
git commit -m "Ajoute le champ marque aux produits"
git push -u origin ajout-champ-marque  # puis ouvrir une Pull Request sur GitHub
```

Une branche par fonctionnalité permet de montrer le travail, de le faire relire et de revenir en arrière facilement.

9. Pour progresser

- **Laravel** (documentation officielle, très claire) : <https://laravel.com/docs> — commencez par *Routing*, *Controllers*, *Blade*, *Eloquent*, *Validation*, *Authorization*.
- **Laracasts** (vidéos) : la série « *Laravel From Scratch* ».
- **Tailwind CSS** : <https://tailwindcss.com/docs> — pour comprendre les classes `px-4`, `rounded-xl` ...
- **Alpine.js** : <https://alpinejs.dev> — les `x-data`, `@click`, `x-show` dans les vues.

Le meilleur exercice : réalisez vous-même la **Recette A** (champ « Marque ») de bout en bout, sur une branche Git. Vous aurez touché toutes les couches du projet.